ENGR-E321
Advanced Cyber-Physical Systems

Section: 10231

Lab 4: PID Line Following

Due: March 9th, 2026

By: Jaiden Medina

Brady Adams

Table of Contents

1. Overview 3
   1.1 Purpose 3
   1.2 Background Information 3
2. System Implementation 4
   2.1 Control Loop Structure 4
3. PID Controller Implementation 4
4. Used PID Gains 5
5. PID Terms 5
   5.1 Proportional Term (P) 5
   5.1 Integral Term (I) 5
   5.3 Derivative Term (D) 6
6. Motor Speed Control 6
7. Controller Performance Analysis 6
   7.1 P-Controller Performance 7
      7.1.2 Discussion 7
   7.2 PD-Controller Performance 8
      7.2.1 Discussion 8
   7.3 PID-Controller Performance 8
      7.3.1 Discussion 9
   7.4 Recommended Controller for Lab Specifications 9
8. Curve Handling Strategy 10
9. PID Algorithm Pseudocode 10
10. Data Logging 12
11. Comparison of P, PD, and PID Controllers 13
   11.1 P-Controller 13
   11.2 PD-Controller 13
   11.3 PID-Controller 14

12. Relationship to the Line Sensor System from Lab 3 14
13. Fastest Line Following Configuration 15
14. Challenges Encountered 15
14.1 Sensor Calibration 15
14.2 Oscillation 15
14.3 Integral Windup 15
14.4 High Speed Instability 15
15. Conclusion 16

1. Overview

1.1 Purpose

In this lab a PID based line-following controller was implemented on the Romi driving robot. The purpose of this project was to design a control system that continuously reads the line sensor array, using the infrared line sensors found wired to the Romi robot on the bottom end, which then computes an error relative to the center of the track, and adjusts the speed of the motors to keep the robot aligned with the black line throughout the course.

1.2 Background Information

The controller operates as a feedback control loop, where the robot measures the line position using the QTR sensor array, then computes an error relative to the center of the line, uses a PID controller to compute a correction value, and finally, adjusts the left and right motor speeds accordingly.

The PID loop runs periodically and updates the motor commands so the robot can react to deviations of the line path. In this lab specifically, the requirements included logging error and wheel speeds so the performance of different control strategies (Specifically P, PD, and PID) could be analyzed.

2. System Implementation

2.1 Control Loop Structure

The system operates in three modes. The robot switches between these three modes, those being:

WAIT_CALIBRATE

WAIT_START

RUNNING

WAIT_CALIBRATE prompted the robot to wait for calibration. WAIT_START prompted the robot to wait to begin line following. And RUNNING prompted the robot to have the PID controller actively drive the robot.

The robot first calibrates the sensors so that black and white surfaces can be correctly detected. After calibration, the robot waits until the user presses the A button to start the line follower. During the **RUNNING mode**, the PID controller executes periodically (10 times a second) to update the motor commands.

3. PID Controller Implementation

The PID controller is implemented using the function "runLineFollower()". The function performs each step of the control loop, those being:

1. Read the line position
2. Compute the tracking error
3. Computing the P, I, and D
4. Generate a correction signal
5. Adjust the left and right motor speeds accordingly
6. Log the results

The way the error was computed was by using the equation below:

$$Error = Center - Position$$

As defined in the previous lab, the center was equal to 2500 in the sensors. When the line was under the Romi robot, the error would be as close to 0 as possible and when the line happens to be to the right and left, the error would correspondingly be positive or negative.

4. Used PID Gains

The gains used in the controller were:

- Kp = 0.04
- Ki = 0.0005
- Kd = 0.02

These values were tuned experimentally when observing the robots behavior and analysis of the logged data

5. PID Terms

5.1 Proportional Term (P)

The proportional term responds directly to the error, as seen below:

$$P = Kp * Error$$

If the Romi robot deviates greatly from the line center, the proportional term (p) will produce a larger correction to push it back to the center. The effect that this has on the robot is that it helps the robot steer toward the line, it has a faster response to deviations, but if the deviation is to large, the Romi robot will start to oscillate.

5.1 Integral Term (I)

The integral term accumulates the error over time, as seen below:

$$Integral += Error$$

$$I = Ki * Integral$$

The purpose of the integral term (I) is to correct the steady-state error, where the robot may consistently dive slightly off the center of the line.

The integral accumulation can grow too large throughout the activation, so the program used includes anti-windup limits, as seen below:

if integral > 10000 → clamp
if integral < -10000 → clamp

Having this clamp prevented the controller from producing extremely large corrections, therefore overcorrecting.

5.3 Derivative Term (D)

The derivative term reacts to how fast the error is changing, as seen below:

$$ErrorChange = Error - lastError$$

$$D = Kd * errorChange$$

This helped reduce the oscillations produced by the P and I terms and improve stability. The effect this had on the Romi robot was that it dampened oscillation, helped the Romi robot anticipate turns, and improved the recovery speed after disturbances occurred.

6. Motor Speed Control

The final correction value was calculated by adding the P, I, and D terms together, as seen below:

$$Correction = P + I + D$$

The correction was then therefore used to modify the motor speeds, which can be seen below:

$$leftSpeed = driveSpeed - correction$$

$$rightSpeed = driveSpeed + correction$$

The leftSpeed corresponds to the amount of torque produced on the right motor, and vice versa for the rightSpeed variable. This differential speed causes the robot to steer toward the line.

7. Controller Performance Analysis

In order to evaluate the performance of the line-following controller, three control strategies were tested:

- Proportional (P)
- Proportional-Derivative (PD)
- Proportional-Integral-Derivative (PID)

For each controller configuration, the robot followed the track while logging:

- Line tracking error
- Left motor speed
- Right motor speed
- Time

The logged data was exported and graphed as error vs time. These graphs show how the robot responds when it encounters disturbances such as sharp turns in the track. The main metric used for comparison is how quickly the robot returns to the line after a disturbance.

7.1 P-Controller Performance

The P controller used only proportional feedback, meaning the correction applied to the motors was directly proportional to the line tracking error. The graph can be seen below:

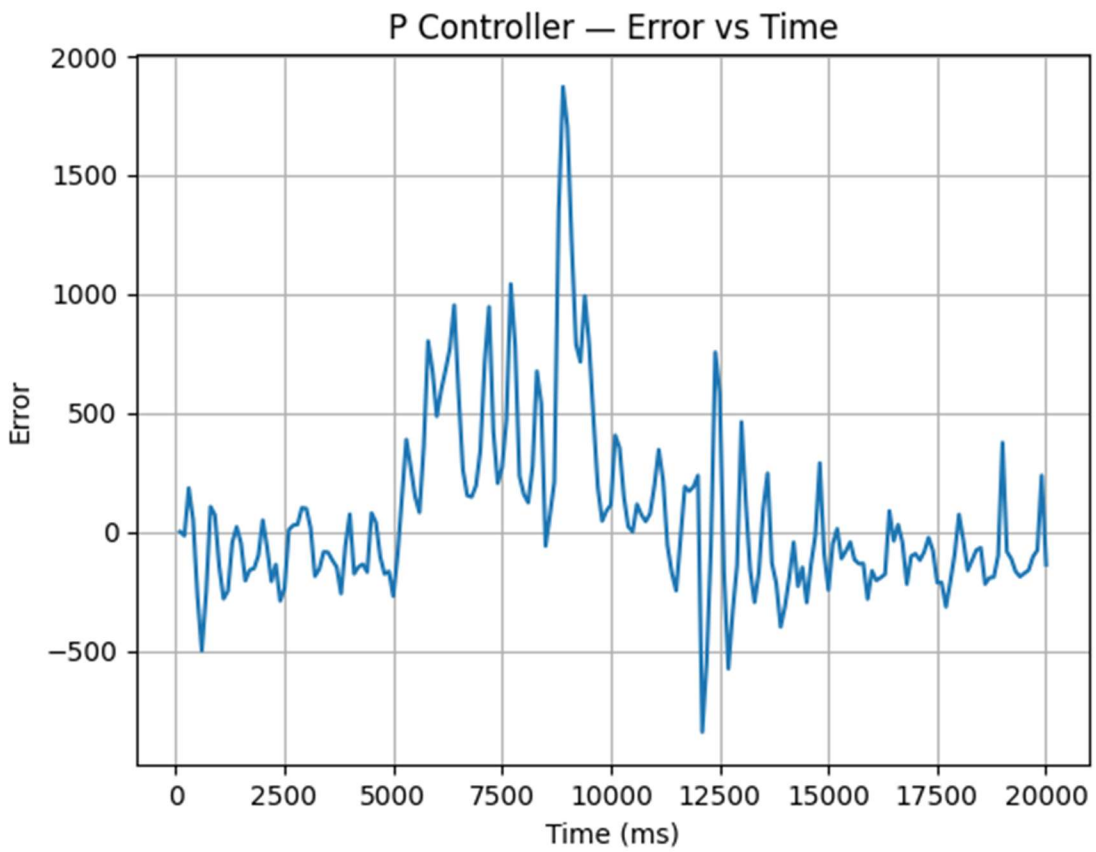

Figure 1: Error vs Time for P Controller

7.1.2 Discussion

The P controller was able to follow the line but showed noticeable oscillations when the robot encountered turns. Because the controller only reacts to the current error, it tends to overshoot the line and then correct back repeatedly.

In the graph, this can be seen in the large spikes in error around the turning sections of the track. The robot takes longer to settle back to zero error after these disturbances.

7.2 PD-Controller Performance

The PD controller adds a derivative term, which responds to the rate of change of the error. This helps reduce oscillations and improves stability. The graph can be seen below:

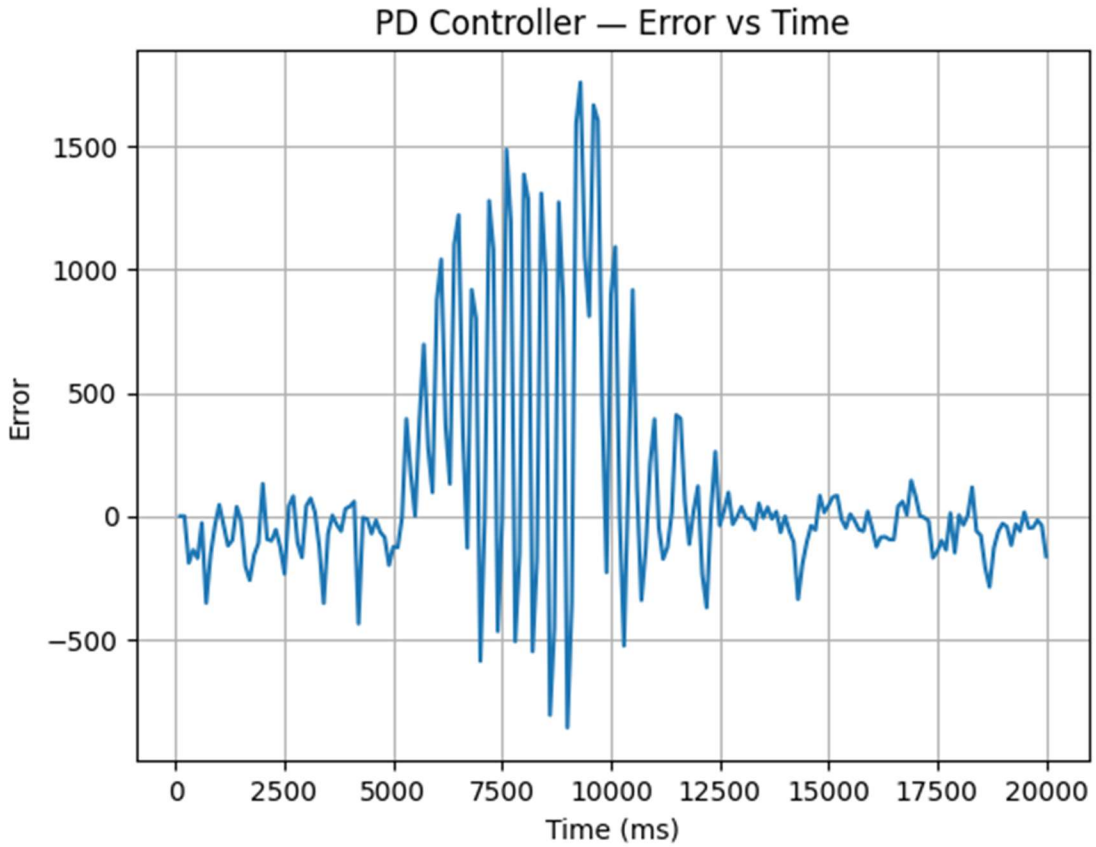

Figure 2: Error vs Time for PD Controller

7.2.1 Discussion

The PD controller produced significantly smoother tracking behavior compared to the P controller. The derivative term dampens rapid changes in error, preventing the robot from overshooting the line as aggressively.

From the graph, the error spikes during turns are still present but settle more quickly. The robot returns to the line faster after disturbances, which improves overall tracking performance.

7.3 PID-Controller Performance

The PID controller includes proportional, integral, and derivative terms. The integral term accumulates error over time and helps eliminate small steady-state errors. The graph can be seen below:

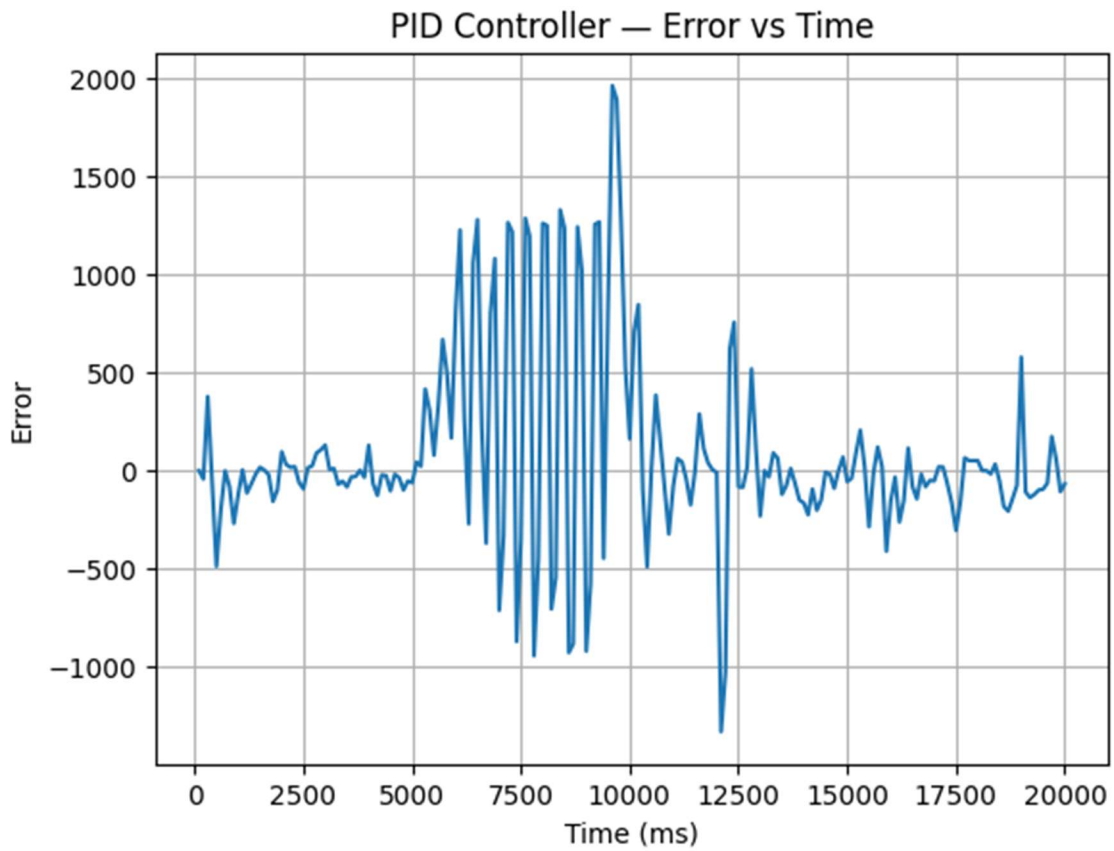

Figure 3: Error vs Time for PID Controller

7.3.1 Discussion

The PID controller was able to maintain accurate line tracking by correcting both instantaneous error and accumulated error. However, the integral component can sometimes slow the controller's response when large errors occur suddenly.

In the graph, the robot still experiences large spikes when encountering sharp turns, but the controller eventually stabilizes the error. Compared to the PD controller, the recovery time was slightly slower in some cases due to the influence of the integral term.

7.4 Recommended Controller for Lab Specifications

The goal of the project was to achieve the fastest reliable line-following robot. Based on the logged data and observed robot behavior:

- The P controller reacted quickly but produced large oscillations.
- The PID controller improved long-term accuracy but sometimes slowed response during sharp turns.
- The PD controller provided the best balance between speed and stability.

Because the PD controller reduced oscillations and returned to the line quickly after disturbances, it produced the fastest stable line-following performance.

8. Curve Handling Strategy

The Romi robot also adjusts its speed depending on how large the error is. The Romi robot detects a sharp turn and prevents overshooting the line using this portion of the program:

```
if abs(error) > 1200
    reduce speed more
else if abs(error) > 600
    reduce speed slightly
```

9. PID Algorithm Pseudocode

The pseudocode below accurately reflects the structure of the program used:

```
Initialize sensors
Initialize motors
Set PID gains

mode = WAIT_CALIBRATE

loop forever:

  if Button A pressed:

    if mode == WAIT_CALIBRATE
      calibrate sensors
      print calibration values
      mode = WAIT_START
```

```
    else if mode == WAIT_START
        reset PID variables
        reset logging arrays
        start timer
        mode = RUNNING

    else if mode == RUNNING
        stop motors
        print logged data
        mode = WAIT_START


if mode == RUNNING:

    if PID_PERIOD has elapsed:

        read line position

        error = CENTER - position

        P = Kp * error

        integral += error
        clamp integral

        I = Ki * integral
```

```
errorChange = error - lastError
D = Kd * errorChange

correction = P + I + D

driveSpeed = baseSpeed

if abs(error) large
   reduce driveSpeed

leftSpeed  = driveSpeed - correction
rightSpeed = driveSpeed + correction

clamp motor speeds

set motor speeds

log time, error, leftSpeed, rightSpeed

lastError = error
```

10. Data Logging

To track how well the Romi robot was performing, the following values were collected:

- Time (ms)
- Error
- Left Motor Speed
- Right Motor Speed

The values were stored in arrays within the program – logTime[], logError[], logLeft[], logRight[].

The logging system captured 200 samples, 10 per second, which provided enough for analysis of controller behavior over time.

When the robot stops, the log is printed to the serial monitor within Arduino IDE and saved for analysis.

11. Comparison of P, PD, and PID Controllers

Three different controller configurations were tested, P-Controller, PD-Controller, and PID-Controller.

11.1 P-Controller

The configuration of the P-Controller included:

- Ki = 0
- Kd = 0

The characteristics of the P-Controller observed were that:

- The robot reacted quickly to error
- Stronger oscillation around the line
- Larger overshoots when entering turns
- Error spikes were common

This can be seen in the data logged as:

- Large error magnitudes
- Frequent Corrections

This behavior is expected because having just proportional control alone cannot dampen oscillations.

11.2 PD-Controller

The configuration of the PD-Controller included:

- Ki = 0
- Kd > 0

The characteristics of the PD-Controller observed were that:

- Much smoother behavior
- Reduced oscillations
- Faster recovery after turns

The derivative term helped stabilize the robot by reducing the sudden changes in direction. This controller produced more stable response when navigating curves.

11.3 PID-Controller

The configuration of the PID-Controller included:

- Kp > 0
- Ki > 0
- Kd > 0

The characteristics of the PID-Controller observed were that:

- Slightly smoother tracking in some sections
- Corrects long-term drift
- Aggressive turn response wasn't perfect

The integral term improved accuracy but sometime recovery slowed is accumulated error was too large.

12. Relationship to the Line Sensor System from Lab 3

Lab 3 focused on installing and characterizing the QTR line sensor array and developing software capable of accurately measuring the position of a line relative to the robot. The sensor system used the six center elements of the array to determine the line position, where the output value represents the displacement of the line from the center of the sensor array. Positive values indicate the line is to the right of center, while negative values indicate it is to the left.

The line position measurement developed in Lab 3 serves as the feedback signal for the control system implemented in Lab 4. During each control cycle, the robot reads the line position from the sensor array and computes a tracking error relative to the desired center position. This error becomes the input to the PID controller, which determines how the left and right motor speeds should be adjusted to steer the robot back toward the center of the line.

Because the sensor system was calibrated and characterized in Lab 3, the line position measurement provides a reliable representation of the robot's deviation from the track. This allows the PID controller to make accurate corrections and maintain stable line tracking. In this way, Lab 3 established the measurement subsystem, while Lab 4 implemented the closed-loop control system that uses those measurements to control the robot's motion.

13. Fastest Line Following Configuration

The fastest configuration, but sometimes less accurate in the long-term, was the PD-Controller.

This was because due the PD loop reduced oscillation and overshoot which allowed it to react faster to turns, had less oscillation and more stable steering compared to the P-Controller.

On the contrary, the PID-Controller provided better steady-state accuracy, but then the integral term sometimes slowed down the recovery during very sharp turns.

This allowed the PD-Controller to be faster.

14. Challenges Encountered

There were several challenges that occurred while creating the program and tuning the robot. These challenges can be seen below.

14.1 Sensor Calibration

The proper calibration was critical to the success of the Romi robot. If the sensors weren't moved enough during the calibration process, the robot would produce incorrect line position readings.

14.2 Oscillation

With large proportional gains the robot oscillated heavily around the line. This required reducing Kp or adding derivative control.

14.3 Integral Windup

Without limiting the integral term, the controller could accumulate large errors that caused large steering corrections. Anti-windup limits were added to solve this.

14.4 High Speed Instability

At higher base speeds the robot overshot turns. This was solved by reducing speed when the error became large.

15. Conclusion

This lab demonstrated the implementation and tuning of a PID-based control system for a line-following robot. The robot continuously measures the line position using the QTR sensor array and computes an error relative to the center of the robot. This error is processed by the PID controller, which generates corrective motor commands that steer the robot back toward the track.

Testing different controller configurations showed clear performance differences. A proportional controller alone was able to follow the line but produced noticeable oscillations and slower recovery when the robot encountered turns. Adding derivative control significantly improved stability by damping oscillations and reducing overshoot, resulting in smoother and faster tracking. While the full PID controller helped reduce small steady-state errors, the PD controller ultimately provided the best balance between responsiveness and stability for achieving the fastest reliable line-following performance.

The sensor module developed in Lab 3 provided the accurate line position measurements required for this control strategy. By integrating that sensing system with a feedback controller, the robot was able to continuously adjust its motion based on real-time sensor data and maintain stable tracking of the line throughout the course.